A Cloud-Native Serverless Architecture Deployed on AWS

1 Introduction

The evolution of Cloud Computing has led to a transition from traditional server-based architectures (IaaS) towards *Serverless* and *Function-as-a-Service* (FaaS) models. These paradigms relieve developers from infrastructure management (a *NoOps* approach) and allow for cost optimization through a *pay-per-use* model. This project implements a 100% serverless *event-driven* data processing pipeline on Amazon Web Services (AWS), with the goal of practically demonstrating the core concepts of the course, such as: *Cloudonomics*, *Rapid Elasticity*, service decoupling, and security implementation via the *Least Privilege* principle.

2 System Architecture

2.1 Overview

Architecture flow:

Local Script → Amazon S3 → AWS Lambda → Amazon DynamoDB.

The system architecture follows a reactive (event-driven) pipeline. A local Python script simulates a massive workload (e.g., IoT devices or user uploads) by sending concurrent files to an Object Storage (S3). The arrival of the data acts as an asynchronous trigger to launch the compute layer (Lambda), which processes the file metadata and persists it in a NoSQL database (DynamoDB). The entire flow is monitored in real-time by an observability system (CloudWatch).

2.2 Components

Amazon S3: Acts as a *Data Lake* for raw file ingestion. It is configured to intercept the object creation event (s3:PutObject) and emit a notification that "wakes up" the compute layer.

AWS Lambda: A *stateless* compute microservice. It receives the JSON event payload from S3, extracts the relevant metadata (bucket name, file key, size), and generates a structured record with a unique UUID.

Amazon DynamoDB: A fully managed NoSQL database used for the structured and permanent storage of the metadata processed by Lambda, acting as the state persistence layer.

AWS IAM: Identity and Access Management handles the authorization and authentication of identities (both human and machine) involved in the architecture, ensuring access security.

Amazon CloudWatch: The *observability* service that collects execution logs and generates time-series metrics to evaluate the system's performance and parallelism level.

3 AWS Serverless Implementation

3.1 Event-Driven Execution

Unlike *polling* models where a server must constantly query a queue, the implemented system is purely event-driven. Computing occurs solely in response to an external stimulus, ensuring that compute resources never remain in an *idle* state.

3.2 Security and Least Privilege

System security was ensured by strictly applying the Least Privilege principle via AWS IAM. For the local client responsible for data ingestion, an *inline* JSON policy was created, allowing the s3:PutObject action exclusively on the specific target bucket. Concurrently, the Lambda function was assigned an *Execution Role* restricted to reading from S3, writing to the specific DynamoDB table, and creating log streams in CloudWatch.

PermessiUploadScriptLocale Copia JSON Modifica

```
1  {
2      "Version": "2012-10-17",
3      "Statement": [
4          {
5              "Sid": "PermessoUploadProgettoCloud",
6              "Effect": "Allow",
7              "Action": [
8                  "s3:PutObject"
9              ],
10             "Resource": [
11                 "arn:aws:s3:::progetto-cloud-calloni-input/*"
12             ]
13         }
14     ]
15 }
```

4 Non-Functional Properties

4.1 Rapid Elasticity

Automatic horizontal scalability is the key property demonstrated by the system. Upon the arrival of dozens of simultaneous requests, the cloud provider automatically provisions multiple independent containers (microVMs) to execute the Lambda function in parallel, processing the workload without latency and destroying the resources as soon as execution finishes.

4.2 Cloudonomics and Cost Efficiency

The project fully demonstrates the "laws of Cloudonomics" discussed during the course. The architecture involves no fixed allocation or maintenance costs (zero *CapEx* and a drastic reduction in *OpEx*). The economic impact occurs only during the actual code execution, billed in milliseconds of consumed computational time.

4.3 High Availability and Fault Tolerance

By exclusively leveraging managed services (PaaS/SaaS) from the cloud provider, the infrastructure natively inherits high availability. S3 and DynamoDB automatically replicate data across multiple *Availability Zones* within the same *Region*, ensuring fault tolerance without the developer needing to configure *Active/Standby* architectures.

5 Demonstration and Results

During the live demonstration:

- The system starts from a "clean" state (no files on S3, empty DynamoDB table).
- A local Python script is executed to generate a sudden traffic spike, uploading 50 files in parallel (using a *ThreadPoolExecutor* configured with 10 threads) to the S3 bucket.
- The S3 dashboard shows the 50 files successfully ingested into the storage.

- The DynamoDB dashboard is queried, demonstrating how the table was instantly populated with 50 newly processed records.

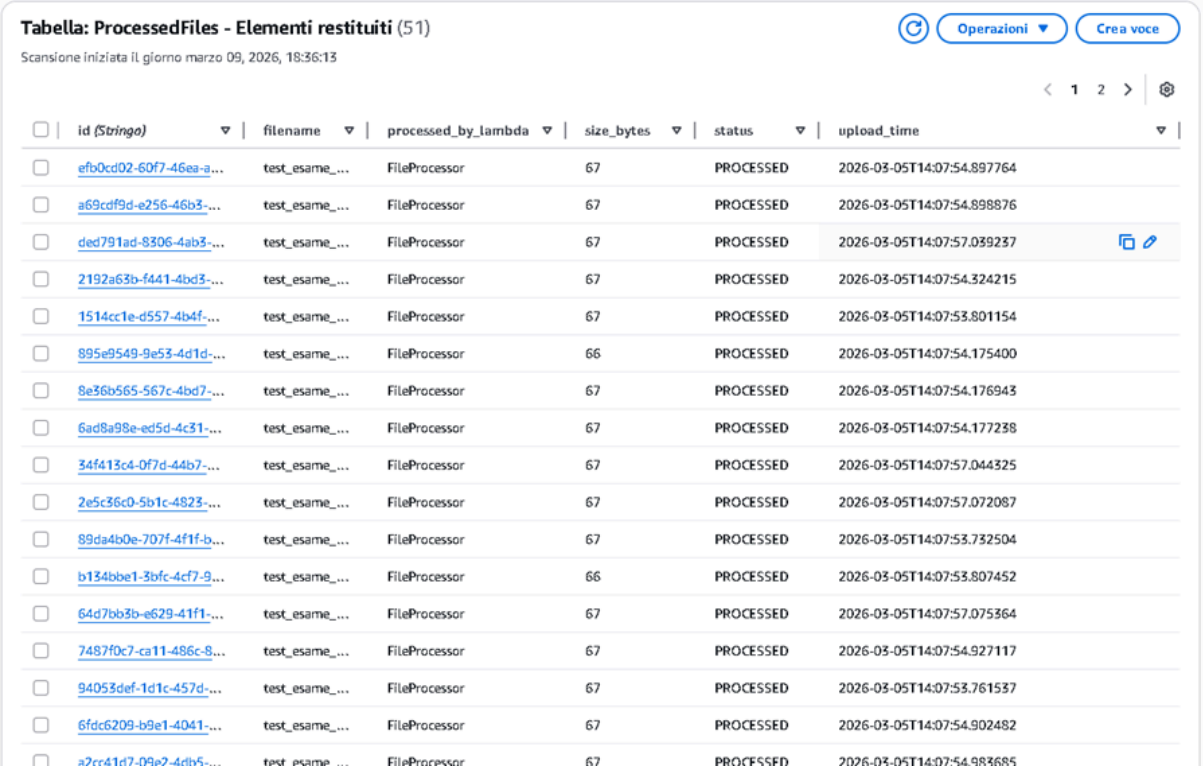


Tabella: ProcessedFiles - Elementi restituiti (51)

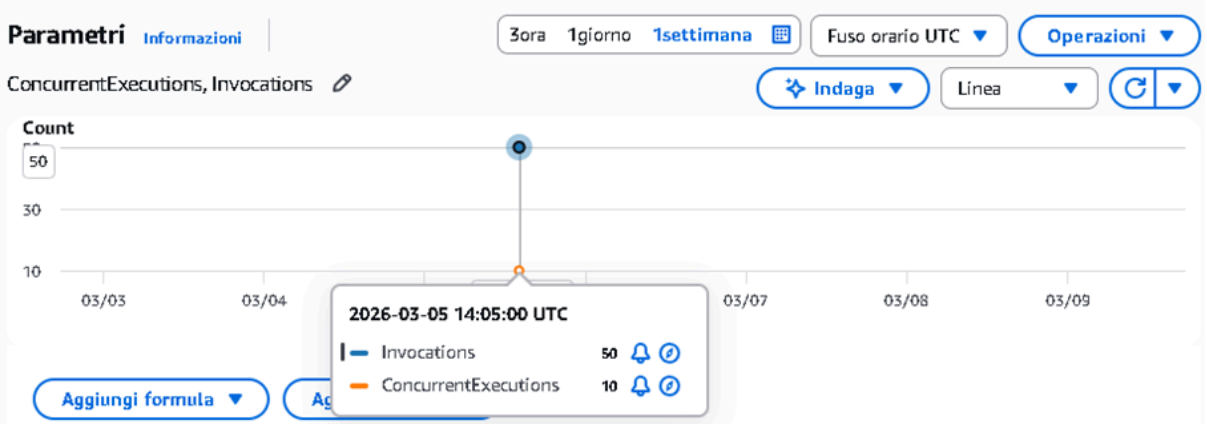
Scansione iniziata il giorno marzo 09, 2026, 18:36:13

Operazioni | Crea voce

1 2

id (Stringa)	filename	processed_by_lambda	size_bytes	status	upload_time
efb0cd02-60f7-46ea-a...	test_esame_...	FileProcessor	67	PROCESSED	2026-03-05T14:07:54.897764
a69cdf9d-e256-46b3-...	test_esame_...	FileProcessor	67	PROCESSED	2026-03-05T14:07:54.898876
ded791ad-8306-4ab3-...	test_esame_...	FileProcessor	67	PROCESSED	2026-03-05T14:07:57.039237
2192a63b-f441-4bd3-...	test_esame_...	FileProcessor	67	PROCESSED	2026-03-05T14:07:54.324215
1514cc1e-d557-4b4f-...	test_esame_...	FileProcessor	67	PROCESSED	2026-03-05T14:07:53.801154
895e9549-9e53-4d1d-...	test_esame_...	FileProcessor	66	PROCESSED	2026-03-05T14:07:54.175400
8e36b565-567c-4bd7-...	test_esame_...	FileProcessor	67	PROCESSED	2026-03-05T14:07:54.176943
6ad8a98e-ed5d-4c31-...	test_esame_...	FileProcessor	67	PROCESSED	2026-03-05T14:07:54.177238
34f413c4-0f7d-44b7-...	test_esame_...	FileProcessor	67	PROCESSED	2026-03-05T14:07:57.044325
2e5c36c0-5b1c-4823-...	test_esame_...	FileProcessor	67	PROCESSED	2026-03-05T14:07:57.072087
89da4b0e-707f-4f1f-b...	test_esame_...	FileProcessor	67	PROCESSED	2026-03-05T14:07:53.732504
b134bbe1-3bfc-4cf7-9...	test_esame_...	FileProcessor	66	PROCESSED	2026-03-05T14:07:53.807452
64d7bb3b-e629-41f1-...	test_esame_...	FileProcessor	67	PROCESSED	2026-03-05T14:07:57.075364
7487f0c7-ca11-486c-8...	test_esame_...	FileProcessor	67	PROCESSED	2026-03-05T14:07:54.927117
94053def-1d1c-457d-...	test_esame_...	FileProcessor	67	PROCESSED	2026-03-05T14:07:53.761537
6fdc6209-b9e1-4041-...	test_esame_...	FileProcessor	67	PROCESSED	2026-03-05T14:07:54.902482
a2cc41d7-09e2-4db5-...	test_esame_...	FileProcessor	67	PROCESSED	2026-03-05T14:07:54.983685

- CloudWatch metrics are analyzed: the *Invocations* metric (set to Sum) shows a peak of 50 executions in the current minute, while the *ConcurrentExecutions* metric (set to Max) visually proves the parallel provisioning of 10 containers to handle the load.



6 Conclusion

The project allowed for the practical application of theoretical Cloud Computing concepts in a real-world production environment. The implementation demonstrated how adopting a Cloud-Native Serverless architecture guarantees total service decoupling, instantaneous scalability under load, and optimized financial management. It provided hands-on validation of the shift from managing physical and virtual servers to the purely logical management of applications and permissions.